Scalpel Master Handbook

Status: long-term product handbook. Current codebase date: 2026-06-04. Current implementation: TypeScript MCP server with local stdio transport and 14 file tools. Target product: Apache-2.0 Rust-core local operations engine with MCP and CLI surfaces.

This handbook describes what Scalpel is meant to become. It also names what exists today so future planning does not confuse aspiration with shipped behavior.

---

1. Vision

Scalpel is a serious local operations engine for developers, agents, infrastructure teams, and power users.

It should safely inspect, plan, apply, undo, and journal file and codebase operations across local workspaces and opt-in full-volume scopes. The product is agent-native first, but the CLI is equally important: it is the inspection, debugging, recovery, and power-user control plane.

The long-term promise:

- coding agents get precise, deterministic, machine-readable file operations
- humans get a fast CLI for inspection, planning, applying, undoing, and recovering operations
- infrastructure workflows get plan/apply discipline, semantic validation, and durable recovery
- massive local datasets become searchable, cataloged, deduplicated, movable, and transformable without requiring cloud latency or network access

Scalpel is not intended to stay a small JavaScript MCP wrapper. The TypeScript server is the first interface and proving ground. The core must become Rust.

---

2. Current Reality

Current code provides:

- local MCP server over `stdio`
- root-confined path policy
- 14 public tools
- exact text edits
- line and marker edits
- dry-run diffs for most edit tools
- optimistic concurrency for most existing-file mutations
- best-effort atomic replacement through temp file plus rename

- Vitest unit and stdio integration coverage

Current code does not yet provide:

- Rust core
- CLI surface
- plan/apply commit tokens
- durable journal
- undo stack
- persistent index
- streaming large-file edits
- binary-safe operations
- parser-aware edits
- secret redaction
- native performance layer
- volume mode
- daemon or watcher
- crash recovery

The current implementation is useful as an MVP and contract discovery layer. It is not yet the target architecture.

---

3. Product Principles

Local First

Core indexing, planning, applying, undo, and recovery must work offline.

Network features may enhance validation, documentation lookup, registry metadata, embeddings, policy sync, or cloud-specific dry runs. Network access must never be required for local file safety.

Safe By Default, Powerful On Purpose

Default mode is workspace scoped and conservative.

Volume and filesystem-wide mode are explicit opt-ins. Once enabled, Scalpel may support destructive operations such as tree deletes, overwrites, chmod/chown, symlink and hardlink operations, archive extraction, bulk rewrites, dedupe moves, and cross-tree transforms.

The product should not be artificially toothless. It should be powerful and accountable.

Plan Before Damage

Destructive, high-risk, large, or irreversible operations use a two-phase prepare/apply model.

Prepare computes:

- exact operation graph
- target path set
- expected byte and file impact
- risk tier
- numeric risk score
- preconditions
- rollback or snapshot plan
- hash manifest
- validation steps
- commit token

Apply executes only the exact prepared plan. If the plan manifest does not match, apply fails.

Journal Before Mutation

Every supported mutating operation creates a journal entry before touching disk.

If Scalpel cannot guarantee undo for an operation, the plan must mark that operation irreversible before execution. Irreversible operations require critical confirmation and should produce backup or snapshot alternatives wherever possible.

Agent Native, Human Legible

All critical state must be machine-readable and human-inspectable.

Agents need narrow deterministic tools with structured errors. Humans need CLI commands, readable manifests, explainable risk output, and recovery workflows.

Performance Is A Product Feature

Scalpel should treat large scale as normal:

- single huge files
- huge directory trees
- source workspaces with hundreds of thousands of lines
- log corpora
- package caches
- generated artifacts

- archives
- VM and container volumes
- cloud configuration exports

Small-edit latency matters, but the product identity is max-throughput scans plus safe commits.

---

4. License

Target license: Apache-2.0.

Rationale:

- permissive developer tooling license
- common for infrastructure tools
- explicit patent grant
- friendly to broad adoption in commercial and open-source environments

The repository should include:

- `LICENSE`
- SPDX headers where appropriate
- dependency license inventory in release artifacts

---

5. Distribution Model

The standalone Rust binary is the source of truth.

Target channels:

- standalone release archives
- `cargo install scalpel`
- npm package for MCP and agent ecosystems
- winget for Windows
- Docker image for CI and containers
- later: Scoop, Homebrew, distro packages when maintenance capacity exists

The npm package must wrap or download the native binary. It must not reimplement core behavior in TypeScript.

6. Architecture Decisions

ADR-001: Rust Core Is Mandatory

Decision: scanning, patching, hashing, indexing, globbing, parsing, journaling, and filesystem operations move to Rust.

Rationale:

- native throughput
- memory safety
- cross-platform binary distribution
- strong filesystem and parser ecosystem
- good fit for ripgrep-style search and parallel traversal

TypeScript remains valuable for MCP adapter work and npm ecosystem packaging, but not for the core engine.

ADR-002: CLI And MCP Are Peer Surfaces

Decision: Scalpel exposes both CLI and MCP surfaces.

The CLI is not a debugging afterthought. It is the primary human control plane for plan review, apply, recovery, journal inspection, daemon control, and benchmark execution.

MCP is the agent-native control surface. MCP tools should map to the same engine contracts as CLI commands.

ADR-003: Workspace Scope First, Volume Mode Explicit

Decision: default operations are scoped to a workspace root. Volume mode must be explicit.

Workspace mode protects the common coding-agent use case. Volume mode enables terabyte cataloging, dedupe, archive inspection, and bulk movement without weakening default safety.

ADR-004: Plan Manifests Are Hash Bound

Decision: prepare emits a hash-bound plan manifest and commit token. Apply verifies the manifest before execution.

Later versions may support signed manifests for CI, release, and organization workflows.

ADR-005: Tiered Indexing

Decision: Scalpel uses tiered indexing instead of blindly storing all content.

Index tiers:

- Tier 0: paths, metadata, sizes, modification times, file IDs
- Tier 1: hashes, MIME/type detection, language detection
- Tier 2: snippets and symbol index data
- Tier 3: full text for approved file classes
- Tier 4: embeddings later

The index must speed up massive search while controlling disk growth, secret exposure, and content retention risk.

ADR-006: Formatting Preservation By Default

Decision: parser-backed edits preserve formatting exactly by default.

Formatter-backed rewrites require explicit opt-in through flags, policy configuration, or operation-specific behavior.

Agents need minimal diffs and predictable output.

---

7. Target System Architecture

Target repository shape:

```
crates/
  scalpel-core/
  scalpel-fs/
  scalpel-plan/
  scalpel-journal/
  scalpel-index/
  scalpel-parse/
  scalpel-policy/
  scalpel-secrets/
  scalpel-watch/
  scalpel-cli/
packages/
  scalpel-mcp/
  scalpel-npm/
src/
  current TypeScript MCP server until replaced or wrapped
docs/
  current implementation docs
scalpel-reliability-suite/
  fixtures, crash tests, benchmarks, golden outputs
```

Target runtime components:

- Rust core libraries
- Rust CLI binary
- MCP adapter package that invokes or embeds the Rust engine
- optional workspace daemon for watch/index workflows
- persistent workspace state under `.scalpel/`
- global supervisor state in OS app data directories

---

8. Workspace State

Default workspace state lives inside the workspace:

```
.scalpel/
  config.toml
  journal/
  plans/
  snapshots/
  index/
  locks/
  tmp/
```

Global application data stores:

- user configuration
- cross-root registry
- daemon state
- shared cache
- downloaded native binaries for package wrappers

Scalpel must support `--journal-dir` for custom journal placement.

---

9. Configuration Model

Configuration is layered.

Precedence:

1. CLI flags
2. workspace internal config: `.scalpel/config.toml`
3. repo config: `scalpel.toml`
4. global user config

5. defaults

Project-facing `scalpel.toml` records policy and intended behavior. Internal `.scalpel/config.toml` records workspace state settings and generated local choices.

Example:

```
[scope]
mode = "workspace"
roots = ["."]

[safety]
require_prepare_for_destructive = true
default_retention_days = 30
snapshot_size_cap_gb = 20
snapshot_operation_cap = 200

[index]
enabled = true
store_full_text = false
max_file_bytes_for_text = 2097152

[secrets]
redact_by_default = true
raw_content_requires_opt_in = true

[validation]
offline_required = true
external_tools = ["terraform", "kubectl", "docker", "pnpm", "cargo"]
```

---

10. Policy Model

Scalpel supports:

- static TOML policy
- programmable policy hooks for advanced users and organizations

Policy hooks should be:

- sandboxed
- deterministic where possible
- time bounded
- offline capable
- incapable of silently expanding operation scope

Policy decisions should be captured in plan manifests and journal entries.

11. Risk Model

Every plan receives:

- risk tier: `low`, `medium`, `high`, or `critical`
- risk score: integer from 0 to 100
- reason list
- mitigations
- irreversibility status

Examples:

- low: read-only scan, exact single-file patch with undo snapshot
- medium: multi-file rewrite inside workspace with full rollback
- high: recursive delete with snapshot and bounded scope
- critical: irreversible chmod/chown, volume-wide rewrite, archive extraction with conflicts, cross-root delete

Risk scoring is used by policies, dashboards, agents, and audit workflows.

12. Plan And Apply Model

Prepare command:

```
scalpel plan delete path/to/tree --recursive
```

Prepare writes:

```
.scalpel/plans/<plan_id>.json
.scalpel/plans/<plan_id>.human.md
```

Apply command:

```
scalpel apply <plan_id> --token <commit_token>
```

Apply must verify:

- plan hash
- token validity
- root identity
- policy version

- path set
- expected file IDs or metadata
- expected hashes when required
- journal availability
- snapshot capacity
- validation status

If any checked precondition fails, apply refuses to mutate.

---

13. Recovery Model

Recovery is a first-class subsystem.

Minimum supported state:

- operation journal
- undo stack
- snapshots
- hash manifests
- before/after patches
- resumable interrupted operations
- VCS integration

Hard guarantee:

Supported mutating operations are undoable until configured retention limits are reached.

Default retention:

- 30 days
- 20 GB
- 200 operations

Whichever limit triggers first controls cleanup.

---

14. Secret Handling

Secret detection and redaction are core engine responsibilities.

Secrets are redacted by default in:

- logs

- journals
- index records
- previews
- diffs exposed through MCP
- CLI output

Raw content access requires explicit opt-in. Secret handling is not a UI concern; it belongs in the engine.

---

15. Parser-Aware Editing

First parser-aware formats:

- TypeScript
- JavaScript
- Python
- JSON
- JSONC
- YAML
- TOML
- Markdown
- Terraform/HCL
- Kubernetes YAML

Tree-sitter is the preferred parser substrate where practical because incremental concrete syntax trees are a strong fit for safe local edits.

Parser-backed operations should expose:

- symbol lookup
- insert import
- rename symbol in bounded scope
- update function body
- edit JSON/YAML/TOML key
- replace markdown section
- validate HCL block structure
- validate Kubernetes resource shape

Default behavior preserves formatting exactly.

16. Infrastructure Validation

Scalpel should call external tools when available:

- `terraform`
- `kubectl`
- `docker`
- `gh`
- `az`
- `aws`
- `npm`
- `pnpm`
- `yarn`
- `cargo`
- `python`
- `ruff`
- `mypy`
- `pytest`
- `node`
- `tsc`
- `eslint`

Validation must degrade gracefully when tools are missing or offline.

For critical infrastructure files, semantic validation should be part of planning, not an afterthought.

---

17. MCP Surface

Scalpel should expose many narrow deterministic tools plus a smaller number of orchestration tools.

Narrow tools:

- `stat`
- `read`
- `list_dir`
- `grep`
- `diff`
- `create`
- `patch`
- `batch_edit`
- `insert`

- `delete_range`
- `replace_between_markers`
- `append`
- `prepend`
- `move`

Future orchestration tools:

- `index_workspace`
- `catalog_volume`
- `prepare_operation`
- `apply_operation`
- `prepare_bulk_move`
- `prepare_bulk_delete`
- `semantic_patch`
- `explain_risk`
- `recover_operation`
- `journal_query`
- `validate_infra`

MCP failure payloads must become structured, not text-only.

---

18. CLI Surface

CLI uses Git-style subcommands.

Primary commands:

```
scalpel index
scalpel scan
scalpel plan
scalpel apply
scalpel undo
scalpel journal
scalpel doctor
scalpel mcp
scalpel daemon
```

CLI responsibilities:

- inspect current state
- generate and explain plans
- apply commit tokens

- recover interrupted operations
- inspect journals
- manage daemon and index state
- run benchmarks
- debug MCP behavior

---

19. Indexing Model

Persistent indexes are used for projects and volumes. Ephemeral session indexes are used for temporary work.

SQLite WAL is the default metadata store until benchmarks prove it is insufficient. RocksDB remains a candidate for very high-volume key-value/event workloads.

Index requirements:

- stale results must be marked stale
- stale index data must never drive destructive commits
- index writes must be recoverable
- index rebuilds must not block ordinary safe operations
- secret redaction applies before content leaves the engine

---

20. Daemon And Watcher Model

No daemon is required for early Rust phases.

Add a daemon when watcher-based indexing becomes real.

Final model:

- one daemon per active indexed workspace root
- optional global supervisor
- recursive watch
- change tracking over time
- action triggers only through policy-controlled hooks
- no silent destructive behavior

Watchman is the reference class: recursive file watching, tracked changes, and queryable state.

21. Benchmarks

Benchmark goals:

- index 1 million files without crashing
- search a 100 GB text corpus
- parse and vectorize a 500,000 LOC workspace
- patch a 10 GB log through streaming
- catalog a 5 TB directory tree
- recover cleanly from a killed process during mutation
- detect symlink loops
- handle permission-denied paths without aborting an entire scan

Benchmarks must be reproducible and versioned with fixtures or generators.

---

22. Test Strategy

The reliability suite is part of the product identity.

It should grow into:

- stress fixtures
- golden outputs
- mutation tests
- crash recovery tests
- malformed archive tests
- huge-file tests
- symlink-loop tests
- permission tests
- cross-platform matrix runs
- parser fixture suites
- benchmark harnesses

Test classes:

- unit tests for pure logic
- integration tests for filesystem behavior
- stdio MCP tests
- CLI golden tests
- property tests
- fuzz tests
- crash and recovery tests

- cross-platform tests
- benchmark regression tests

---

23. Threat Model

Primary risks:

- wrong-file write
- lost write
- partial write reported as success
- unrecoverable delete
- plan/apply mismatch
- stale index used for destructive commit
- symlink or reparse-point escape
- archive extraction path escape
- chmod/chown outside planned scope
- secret leakage through logs, previews, journals, indexes, or MCP responses
- race between validation and mutation
- malicious workspace config or policy hook

Scalpel must assume files can change while it is planning and applying.

---

24. Data Loss Standard

The following are serious correctness failures:

- wrong-file write
- lost write
- unrecoverable delete
- partial write reported as success
- plan/apply mismatch
- stale index used for destructive operation
- unexpected symlink traversal
- archive extraction path escape
- chmod or chown outside planned scope

Stale search results are acceptable only if clearly marked stale and never used for destructive commits.

25. Release Roadmap

v0.1: TypeScript MCP MVP

Current state. Local stdio MCP server with 14 tools and initial safety contracts.

v0.2: Contract Stabilization

Make the TypeScript MVP internally consistent:

- structured failure payloads
- lint scope fixed
- full dry-run coverage where possible
- consistent optimistic concurrency
- atomicity wording and tests
- reliability suite cleaned up

v0.3: Rust Core Skeleton

Introduce Rust workspace, shared data models, CLI shell, and engine result contracts.

v0.4: Journaled Plan/Apply

Implement plan manifests, commit tokens, journal entries, snapshots, and undo for a narrow set of file operations.

v0.5: CLI Control Plane

Make CLI the primary human interface for inspect, plan, apply, undo, journal, doctor, and MCP launch.

v0.6: Native Tool Parity

Port core current file operations to Rust and have MCP invoke Rust contracts.

v0.7: Indexing Foundation

Add SQLite-backed tier 0 to tier 2 workspace indexing, query APIs, and stale-state handling.

v0.8: Parser-Aware Editing

Add parser-backed edits for TS/JS, Python, JSON/JSONC, YAML, TOML, Markdown, Terraform/HCL, and Kubernetes YAML.

v0.9: Volume Mode And Bulk Ops

Add opt-in volume cataloging, bulk move/delete/rewrite planning, archive inspection, dedupe planning, and large-file streaming.

v1.0: Stable Local Operations Engine

Release when safety, recovery, CLI, MCP, indexing, packaging, benchmarks, and cross-platform behavior meet stable bars.

---

26. Non-Negotiable Invariants

- no mutation without resolved root scope
- no destructive or large mutation without prepare/apply
- no apply when plan hash differs
- no supported mutation without journal entry first
- no success response after partial failure
- no stale index as authority for destructive apply
- no silent symlink or archive path escape
- no unredacted secret in logs or MCP by default
- no formatter-wide rewrite unless explicitly requested
- no current docs claiming future behavior is shipped

---

27. Long-Term Product Shape

Scalpel should become a local operations substrate for agents:

- precise enough for one-line code edits
- cautious enough for critical infrastructure
- fast enough for terabytes
- recoverable enough to trust
- inspectable enough for humans
- structured enough for agents
- local-first enough to work when cloud latency or network access is unacceptable